OList E-Commerce Dataset

Late deliveries, unhappy customers

Delivery performance and customer satisfaction in 96,470 Olist orders

Final project · Data Analytics Diploma · Nile University

Mirna Y. Khalafallah - October 2026

The project at a glance

96,470
delivered orders analysed

7
linked tables joined

2
machine learning tasks

4
Power BI dashboard pages



Python (pandas, seaborn, scikit-learn, XGBoost) for steps 1 to 5, Power BI (Power Query, DAX) for step 6

Five business questions guided the work

Q1

How did sales and on-time delivery change over time?

EDA

Q2

Which categories and regions matter most?

EDA

Q3

Do late deliveries hurt customer reviews?

EDA

Q4

Can we predict at purchase if an order will be late?

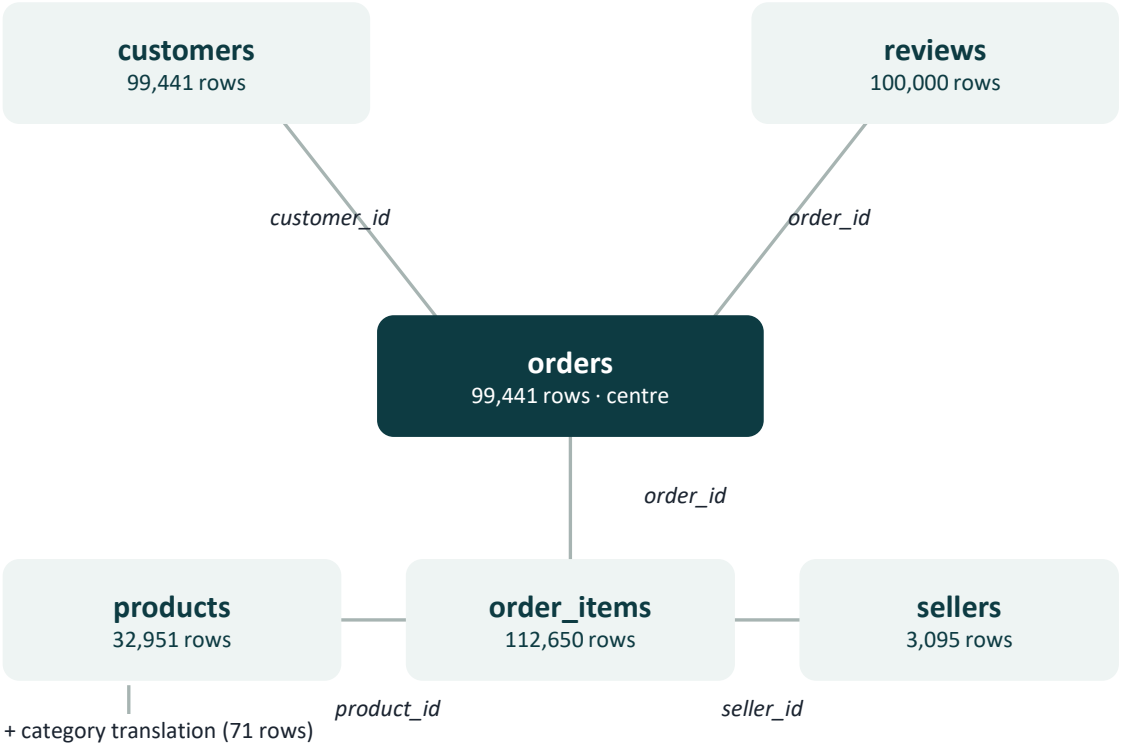
Classification

Q5

Can we predict delivery days better than the promise?

Regression

Seven linked tables, one marketplace



Source
Brazilian E-Commerce Public Dataset by Olist, Kaggle (CC BY-NC-SA 4.0)

Period
Orders from September 2016 to October 2018

What it is
Real, anonymised orders from a marketplace that connects small sellers to large Brazilian online stores

Python

```
orders = pd.read_csv("/content/olist_orders_dataset.csv")    # same for items, customers, sellers,
products, reviews, translation and payments: 8 files
```

How we understood the data

Table	Rows	Columns	Blank cells	Duplicates
orders	99,441	8	4,908	0
items	112,650	7	0	0
customers	99,441	5	0	0
sellers	3,095	4	0	0
products	32,951	9	2,448	0
reviews	100,000	7	146,532	0
translation	71	2	0	0
payments (not used)	103,886	5	0	0

Python output: one row per file, from a loop over the loaded tables

Most blank cells are empty review comments and orders that never arrived

97%

of orders were delivered; 2,965 have no delivery date (Sep 2016 to Oct 2018)

96,096

real customers behind 99,441 customer IDs: a new ID is made for every order

610

products have no category, and 2 have no weight or size

57%

of reviews are 5 stars and 12% are 1 star; 559 orders have two reviews

Python

```
for table in tables: overview.append({"table": table, "rows": df.shape[0], "columns": df.shape[1],
    "blank cells": int(df.isna().sum().sum()), "duplicate rows": int(df.duplicated().sum())})
```

Cleaning kept 96,470 reliable orders

Orders kept

All orders: 99,441

Delivered, with a date: 96,470

2,971 orders removed: cancelled, unavailable, still in transit, or missing a delivery date

Outliers were kept on purpose
8,427 price outliers (IQR rule) are real large purchases, and the slow deliveries are exactly what we study.

Dates
Converted to real dates; 0 deliveries before purchase

Products
English category names; 610 missing set to unknown; 2 weights filled with the median

Reviews
One average score per order (100,000 reviews to 99,441 orders)

Duplicates
None in order, product or seller IDs

Empty cells
15 approval or carrier dates left empty: not used

Python

```
delivered = orders[(orders["order_status"] == "delivered") &
(orders["order_delivered_customer_date"].notna())]
```

New columns turn raw dates into answers

3 · Features

Two targets

late_flag
1 if delivered after the promised date · classification

delivery_days
days from purchase to delivery · regression

Eleven features known at the moment of purchase

customer_state	seller_state	same_state	n_items
n_sellers	total_price	total_freight	total_weight_g
promised_days	purchase_month	purchase_weekday	

Leakage rule: the delivery date, delivery_days and review_score exist only after delivery, so they are never used as features.

Python

```
order_table = sales.groupby("order_id").agg(n_items=("order_item_id", "count"), total_price=("price", "sum"))
df["late_flag"] = (delivered_date > promised_date).astype(int)
```

The final table behind every result

3 · Features

Group	Columns	Used for
Keys (3)	order_id, customer_id, customer_unique_id	Joining, counting customers
Status, dates (6)	order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date	Building targets and trends
Places (4)	**customer_state, seller_state, same_state,** customer_region	Features; region EDA
Order totals (5)	**n_items, n_sellers, total_price, total_freight, total_weight_g**	Features; category EDA
Time (4)	**promised_days, purchase_month, purchase_weekday,** year_month	Features; monthly trend
Review (1)	review_score	EDA only (known after delivery)
Targets (2)	**delivery_days, late_flag**	**What the models predict**

■ 11 model features (known at purchase) ■ 2 targets ■ kept for EDA and checks

96,470 × 25

rows × columns, one row per order

`late_flag`

6.8%

of orders late: 6,534 of 96,470

Classification target · 1 = delivered after the promised date

`delivery_days`

12.5 days

average (median 10) vs 24.4 promised

Regression target · purchase to delivery

Python

```
df.to_csv("/content/Olist_dataset.csv", index=False)
df.shape   # (96470, 25): one row per delivered order, used for EDA, models and Power BI
```

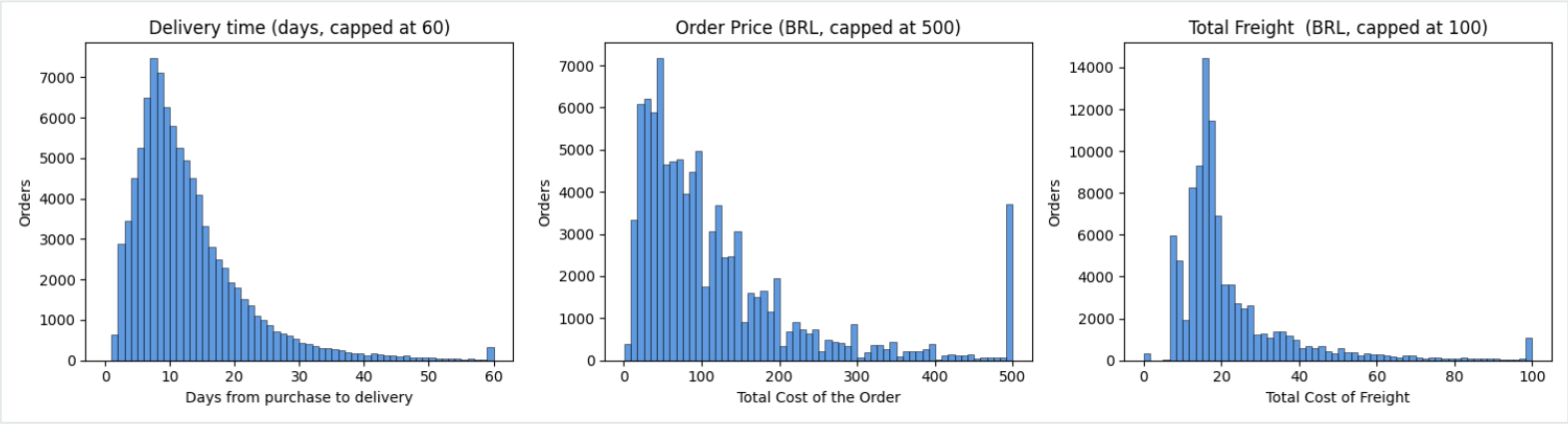
Six EDA questions and what we found

#	Question	How we answered it	Result
1	How are delivery time, order value and freight distributed?	describe() and histograms capped at 60 days	**All right-skewed: median delivery 10 days, mean 12.5**
2	How did orders and on-time delivery change month by month?	groupby(year_month): count orders, mean of late_flag	**Orders grew to about 6,600 a month; on time fell to 81% in March 2018**
3	Which categories bring the most revenue?	groupby(category): sum of price, top 10	**Top 10 bring 62% of 13.2M BRL; health and beauty leads**
4	Which regions get the most late deliveries?	groupby(customer_region): mean of late_flag	**Northeast 12.7% late vs South 5.9%**
5	Do late orders get worse reviews?	average review + crosstab of score shares by status	**2.26 vs 4.28 stars; 54% of late orders get 1 star**
6	How are the features related?	correlation heatmap of the numeric columns	**Delivery days: promised +0.38, same state −0.36, review −0.34**

Python `pandas: describe(), groupby().agg(), pd.crosstab(normalize="index"), corr()` · seaborn charts

Most orders arrive in one to two weeks

Question How are delivery time, order value and freight distributed? **How** describe() and histograms



10 days
median delivery time (slowest: 210 days)

12.5 days
mean delivery: pulled up by a long tail

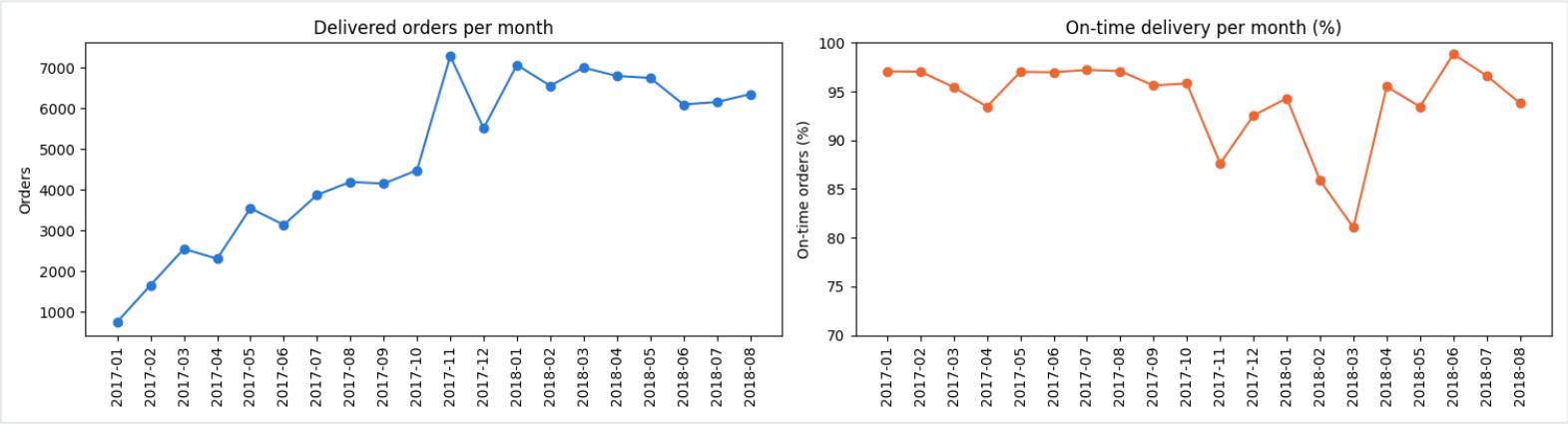
86.5 vs 137
median vs mean order value (BRL)

Python

```
df[["delivery_days", "total_price", "total_freight"]].describe()
sns.histplot(df["delivery_days"].clip(upper=60), bins=60)
```

Sales grew fast, but delays came in waves

Question How did orders and on-time delivery change month by month? **How** groupby(year_month)



7,288
orders in November 2017, the Black Friday peak

93-98%
usual on-time rate in a normal month

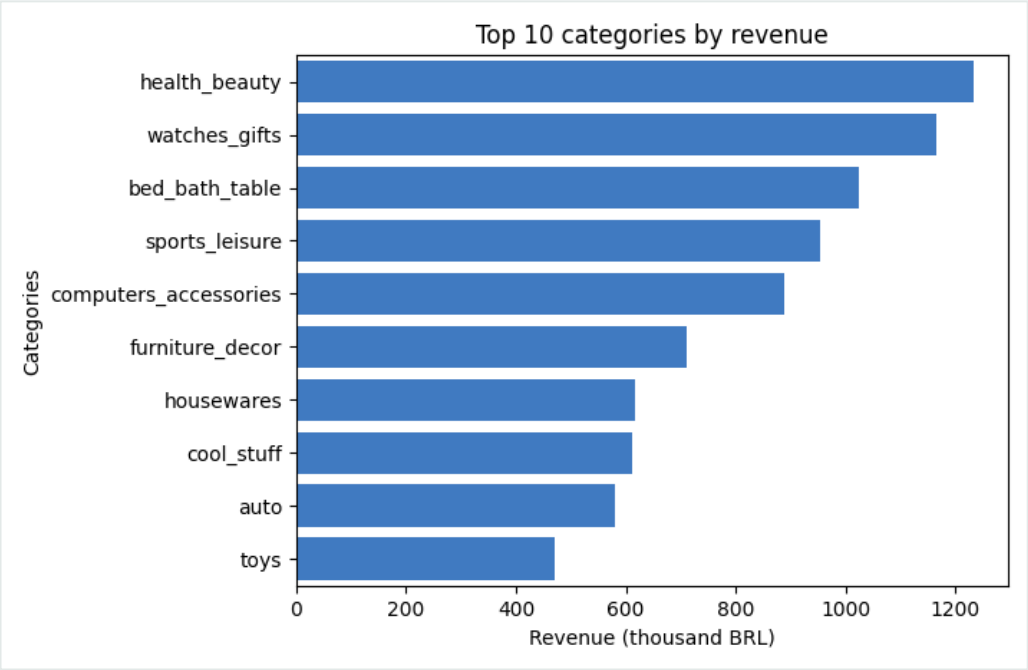
81.0%
on time in March 2018, the worst month

Python

```
monthly = full.groupby("year_month").agg(orders=("order_id", "count"), late_rate=("late_flag", "mean"))
monthly["on_time_pct"] = (1 - monthly["late_rate"]) * 100
```

Ten categories bring 62% of revenue

Question Which categories bring the most revenue? **How** groupby(category), sum of price, top 10



13.2M BRL

total revenue of delivered orders

62%

of revenue from the top 10 of 74 categories

1.23M BRL

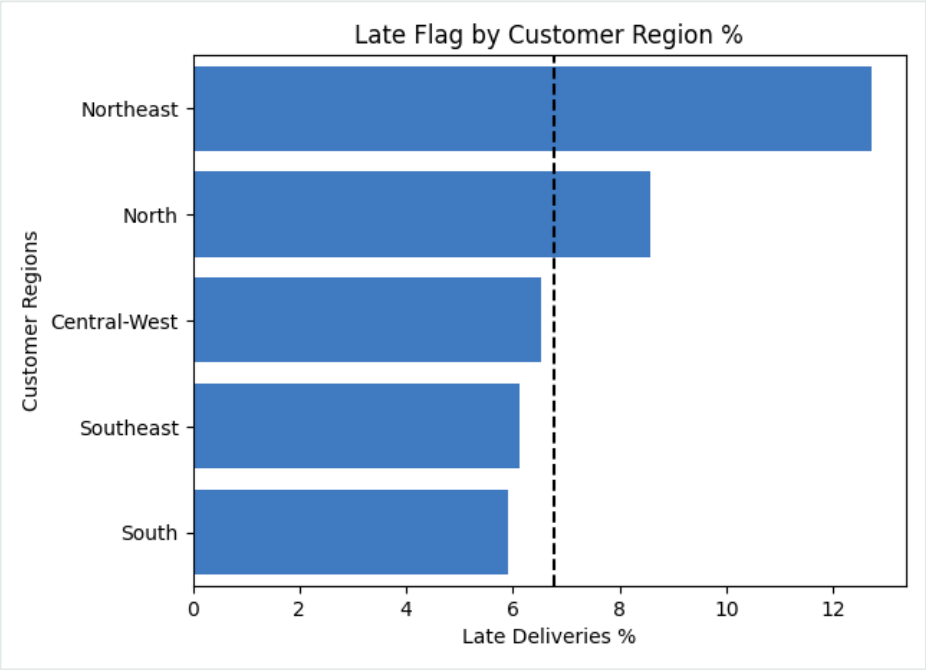
health and beauty, the top category

Python

```
category_revenue = sales.groupby("category")["price"].sum().sort_values(ascending=False)
top_10 = category_revenue.head(10)    # 62% of revenue
```

The Northeast is late most often

Question Which regions get the most late deliveries? **How** groupby(customer_region), mean of late_flag



6.8%

of all delivered orders arrive late (dashed line)

12.7%

late in the Northeast, about twice the South (5.9%)

8.6%

late in the North: long routes from São Paulo sellers

Python

```
by_region = (df.groupby("customer_region")["late_flag"].mean() * 100).sort_values(ascending=False)
```

A late order loses two stars on average

Question Do late orders get worse reviews? **How** average review, and crosstab of score shares



% of orders	1 ★	2 ★	3 ★	4 ★	5 ★
Late	**54.1**	8.8	10.8	10.1	**16.3**
On time	**6.8**	2.7	8.1	20.4	**61.9**

*Python output: pd.crosstab(status, review score, normalize="index") * 100*

4.28 vs 2.26

average review, on time vs late

6.6×

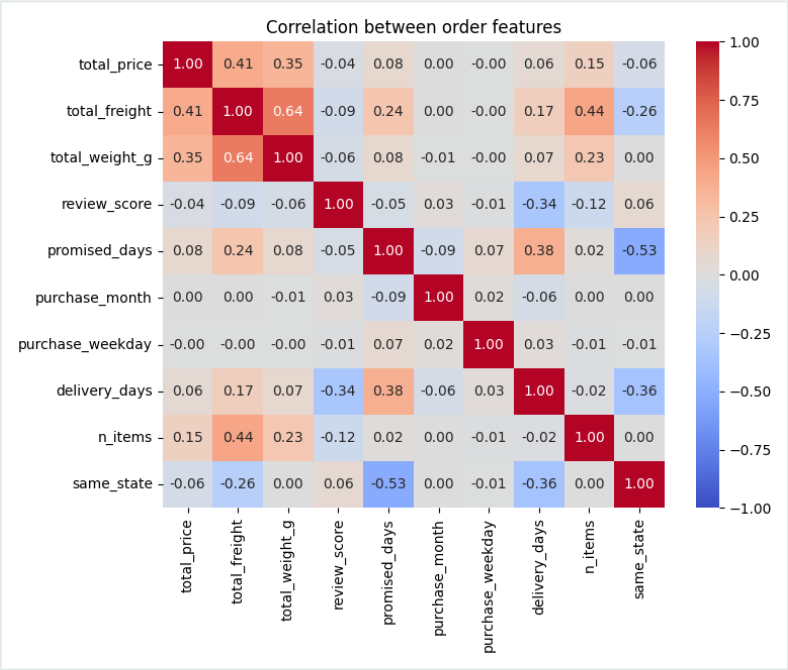
more likely to get 1 or 2 stars when late (62.8% vs 9.5%)

Python

```
status = df["late_flag"].map({0: "On time", 1: "Late"})
pd.crosstab(status, df["review_score"].round().astype(int), normalize="index") * 100
```

No single feature explains delivery time

Question How are the features related? **How** correlation heatmap of the numeric columns



+0.38

longer promises go with longer deliveries

−0.36

same-state orders arrive faster

−0.34

slower deliveries get lower reviews

Python

```
sns.heatmap(df[numeric_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
```

Predicting delivery time within five days

Question Can we predict how many days delivery will take? **How** 3 models vs a baseline, 9 order features, 80/20 split

Model	MAE (days)	RMSE (days)	R^2
Baseline (predict the average)	6.441	9.321	0.000
Linear regression	5.391	8.310	0.205
Random forest	5.219	8.174	0.231
XGBoost	**5.035**	**7.954**	**0.272**

Python output

MAE = average error in days. **RMSE** punishes big misses. **R^2** = share of variation explained.

22% lower

error than guessing the average (5.04 vs 6.44 days)

$R^2 = 0.27$

most variation is not explained

7.95 days

RMSE, well above MAE: rare very slow orders are missed

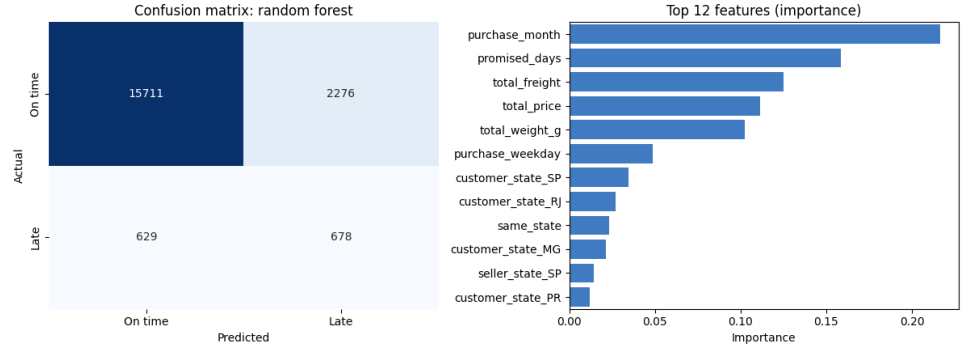
Python

```
Pipeline([("scale", StandardScaler()), ("model", XGBRegressor())]).fit(X_train, y_train)
mean_absolute_error(y_test, y_pred), np.sqrt(mean_squared_error(y_test, y_pred)), r2_score(y_test, y_pred)
```

Flagging late orders at purchase

Question Can we tell at purchase if an order will be late? **How** 11 features incl. states, balanced class weights, stratified split

Model	Recall	Precision	F1	ROC-AUC
Baseline (always on time)	0.000	0.000	0.000	0.500
Logistic regression	0.647	0.124	0.208	0.709
Random forest	**0.519**	**0.230**	**0.318**	**0.788**



Python output

678 of 1,307

late orders caught by the random forest (recall 52%)

3.4×

more reliable warnings than flagging at random (23% vs 6.8%)

2,276

false alarms: suited to cheap actions only

Python

```
RandomForestClassifier(n_estimators=100, min_samples_leaf=10, class_weight="balanced", random_state=42)
recall_score(yc_test, yc_pred), precision_score(...), f1_score(...), roc_auc_score(yc_test, yc_prob)
```

What the classification results mean

The random forest on 19,294 test orders

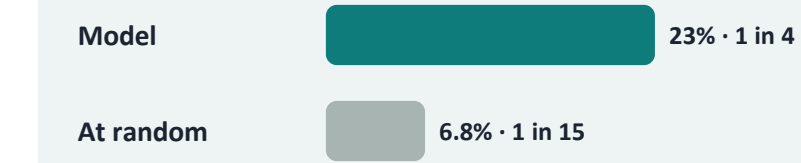
19,294
test orders the model has never seen

2,954
orders it warns about ("will be late")

678
of those warnings were really late

678 ÷ 2,954 = 23% of warnings are right (precision)

Warnings that are really late



23% ÷ 6.8% = 3.4× the model's warnings are 3.4 times more likely to be right (this ratio is called lift)

In everyday terms
Checking 100 random parcels finds about 7 late ones; checking the 100 the model points to finds about 23. Same effort, 3.4 times more problems found. With 3 false alarms per real one, use it for cheap actions.

```python
confusion_matrix(yc_test, yc_pred)   # [[15711, 2276], [629, 678]]  ->  678 / (678 + 2276) = 0.23
```

The models in action on a new order



A new order

- Seller in São Paulo (SP), customer in Bahia (BA)
- 2 items, 150 BRL, freight 25 BRL
- Weight 1.2 kg
- Promised delivery: 25 days
- Bought on a Monday in March

XGBoost regression

13.8 days

expected delivery, 11 days earlier than the 25-day promise (typical error ±5 days)

Random forest classifier

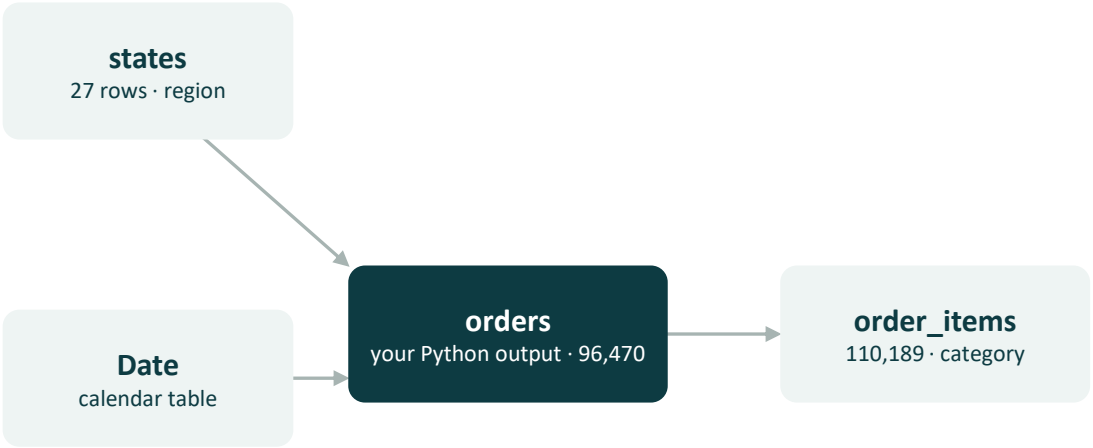
Late risk

flagged as likely late: a cross-state route to the Northeast in March, a month with past delays

Python

```
reg_model3.predict(new_order)[0]   # 13.8 days
classifiers["Random forest"].predict(new_order)[0]   # 1 = late
```

A star schema in Power BI



Filters flow one way: states and Date → orders → order_items (all one-to-many)

Power Query: data types and every new column · DAX: measures only

Key DAX measures

```
Late Rate =
AVERAGE(orders[late_flag])

Late Rate Brazil =
CALCULATE([Late Rate],
REMOVEFILTERS(states))

Share of Status =
DIVIDE([Delivered Orders],
CALCULATE([Delivered Orders],
REMOVEFILTERS(orders[Review
Score])))
```

Power BI Model view: drag orders[order_id] onto order_items[order_id] -> one-to-many, single direction

One number, two tools: the 6.8% late rate

1 · Python creates it

Each order gets late_flag = 1 or 0

```
df["late_flag"] = (
  delivered_date >
  promised_date).astype(int)
df["late_flag"].mean()
# 0.0677
```

2 · One shared file

Python saves the order table once

```
df.to_csv(
  "Olist_dataset.csv",
  index=False)
# 96,470 rows
```

3 · Power BI averages it

The same column, the same rows

```
Late Rate =
  AVERAGE(
    orders[late_flag])
# format: % with 1 decimal
```

6,534 late orders ÷ 96,470 delivered orders = 0.0677 → **6.8%**

Same rows
96,470 delivered orders in both

Same column
late_flag is built once, in Python

Same maths
mean() and AVERAGE of 0/1 = share of 1s

Same filters
Northeast gives 12.7% in both

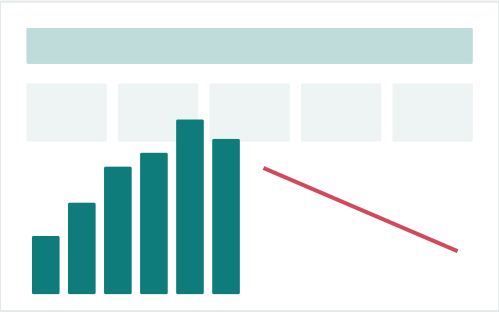
Power BI confirms the Python results

6 · Power BI

Finding	Python	Power BI visual · DAX	Both show
Late rate	`df["late_flag"].mean()`	Card · Late Rate = AVERAGE(orders[late_flag])	**6.8%**
Monthly waves	`groupby("year_month")`	Line chart · On-time Rate = 1 - [Late Rate]	**81.0% in Mar 2018**
Region gap	`groupby("customer_region")`	Bar chart + constant line · Late Rate Brazil = CALCULATE([Late Rate], REMOVEFILTERS(states))	**12.7% vs 5.9%**
Reviews	`pd.crosstab(..., normalize="index")`	Clustered columns · Share of Status = DIVIDE(..., REMOVEFILTERS(orders[Review Score]))	**54% / 62%**
Categories	`groupby("category")["price"].sum()`	Bar chart + Top N filter · Item Revenue = SUM(order_items[price])	**1.23M, top 10 = 62%**
Delivery time	`clip(upper=60) + histplot`	Column chart · Delivery Days (capped) = List.Min({[delivery_days], 60}) in Power Query	**median 10 days**

What Power BI adds: the same numbers, now filtered by region in one click, plus a new view, Promise vs reality.

Page 1 · Overview



What it shows

96,470 orders, 13.22M BRL revenue and a 6.8% late rate. On-time delivery drops to 81% in March 2018, the same waves Python found (EDA Q2).

Visuals

Region slicer: states[region] as tiles, synced to every page

5 cards: Delivered Orders, Revenue, Late Rate, Avg Delivery Days, Avg Review

Column chart: MonthStart × Delivered Orders, Jan 2017 to Aug 2018

Line chart: MonthStart × On-time Rate, y-axis 75% to 100%

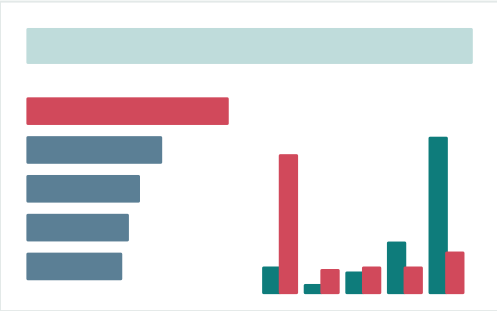
New columns (Power Query)

```
purchase_date =
  Date.From(
  [order_purchase_timestamp])
MonthStart =
  Date.StartOfMonth(
  [purchase_date])
```

New measures (DAX)

```
Delivered Orders = COUNTROWS(orders)
Revenue = SUM(orders[total_price])
Late Rate =
AVERAGE(orders[late_flag])
On-time Rate = 1 - [Late Rate]
Avg Delivery Days =
  AVERAGE(orders[delivery_days])
Avg Review =
AVERAGE(orders[review_score])
```

Page 2 · Delivery and reviews



What it shows

The Northeast's 12.7% bar crosses the 6.8% Brazil line, and 54% of late orders give 1 star, matching EDA Q4 and Q5. Most deliveries take about a week.

Visuals

Bar chart: states[region] × Late Rate, with a dashed Late Rate Brazil constant line

Clustered columns: Review Score × Share of Status, legend = Delivery Status

Column chart: Delivery Days (capped) × Delivered Orders; the 60 bar (60 days or more) in red

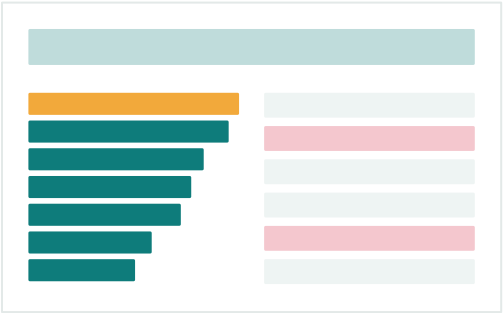
New columns (Power Query)

```
Delivery Status = if [late_flag]
  = 1 then "Late" else "On time"
Review Score =
  Number.Round([review_score], 0)
Delivery Days (capped) =
  List.Min({[delivery_days], 60})
```

New measures (DAX)

```
Late Rate Brazil = CALCULATE(
  [Late Rate],
REMOVEFILTERS(states))
Share of Status = DIVIDE(
  [Delivered Orders], CALCULATE(
  [Delivered Orders], REMOVEFILTERS(
  orders[Review Score])))
Bar Color = IF(MAX(orders[Delivery
  Days (capped)]) = 60, red, blue)
```

Page 3 · Products and states



What it shows

Health and beauty leads with 1.23M BRL and the top 10 bring 62% (EDA Q3). The state table adds detail Python did not show: Rio de Janeiro (12.1%) and Bahia (12.2%) run late.

Visuals

Bar chart: order_items[category] × Item Revenue, Top N filter = 10, top category highlighted

Table: state, name, region, Delivered Orders, Late Rate, Avg Delivery Days, Avg Review

Conditional formatting: red data bars or font when Late Rate > 6.8%

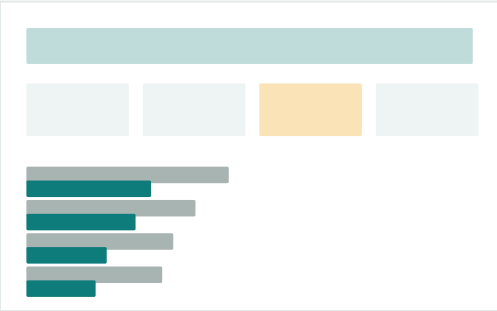
New columns (Power Query)

```
No new columns: category comes
from order_items, state_name and
region from states.csv
```

New measures (DAX)

```
Item Revenue =
  SUM(order_items[price])
Top Category Color = IF(RANKX(
  ALLSELECTED(category),
  [Item Revenue]) = 1, amber, blue)
Formatting rule: Late Rate > 0.068
  → red
```

Page 4 · Promise vs reality



What it shows

Customers are promised 24.4 days but receive orders in 12.5: 11.9 days of padding, and 78.9% arrive a week early. This new view supports the regression result: better estimates are possible.

Visuals

4 cards: Avg Promised Days, Avg Delivery Days, Avg Padding, Early 7+ Rate

Clustered bar: region × Avg Promised Days and Avg Delivery Days

Line chart: MonthStart × both measures

Column chart and table: Early Group × share of orders; Route × days and Late Rate

New columns (Power Query)

```
Early Group = if [promised_days]
  - [delivery_days] < 0 then "Late"
  else if ... <= 4 then "0-4" ...
Early Group Order = 0 to 5
Route = if [same_state] = 1
  then "Same state"
  else "Other state"
```

New measures (DAX)

```
Avg Promised Days =
  AVERAGE(orders[promised_days])
Avg Padding = [Avg Promised Days]
  - [Avg Delivery Days]
Early 7+ Rate = DIVIDE(COUNTROWS(
  FILTER(orders, [promised_days] -
  [delivery_days] >= 7)),
  [Delivered Orders])
```

What the data tells us

Q1 Orders grew ninefold in 2017; on-time delivery is usually 93-98% but fell to 81% in March 2018

Q2 Top 10 categories bring 62% of revenue; the Northeast is late twice as often as the South

Q3 Lateness is the main driver of bad reviews: 2.26 vs 4.28 stars, 6.6× the risk of 1-2 stars

Q4 A random forest catches 52% of late orders; its warnings are 3.4× more reliable than random

Q5 XGBoost predicts delivery time within 5.0 days on average, 22% better than a simple guess

Recommendations for Olist

Prepare for peaks
Add carrier and warehouse capacity before Black Friday and early-year peaks

Fix the Northeast
Regional carriers or sellers closer to North and Northeast customers

Warn risky orders
Use the risk model to message customers before a likely delay

Promise by route
Shorter, data-based delivery dates where routes are reliable

Limitations and next steps



Limitations

- The models only see order data; the real causes of delay (carrier, weather, warehouse load) are missing
- Reviews exist only for delivered orders
- Data covers 2016 to 2018



Next steps

- Add logistics data: carrier, seller history, warehouse workload
- Test the models on future months before using them
- Add payments and seller performance pages



Thank you

Questions and discussion

Python · pandas | seaborn | scikit-learn | XGBoost | Power Query | DAX | Power BI